

Problem 6b: Stateful Conversational Agent with Persistent Memory

This agent is a fully local, modular conversational AI agent designed for an online bookstore (PageTurner Books). It demonstrates how to handle context window limits and provide a personalized experience by utilizing a **Three-Layered Memory Architecture**. The agent successfully remembers facts both within a single conversation and across completely separate sessions using LangGraph, Ollama, and ChromaDB.

Table of Contents

1. What You're Building
 2. Core Concepts
 3. Project Files Overview
 4. The Modular Implementation
 5. The Interactive Demo Explained
 6. Key Design Decisions
 7. Extending the Project
 8. Troubleshooting
-

1. What You're Building

A **stateful customer support chatbot** for an online bookstore (PageTurner Books) named MannKiBot. Unlike a standard chatbot that forgets everything after a conversation ends, MannKiBot:

- Remembers what happened earlier in the current conversation (sliding window + summary)
- Remembers users across completely separate sessions (ChromaDB vector store)
- Can correctly answer "What did I order last time?" in a new session without being told

The scenario that proves it works: **Session 1** — You log in as a new customer (e.g. `mohit_47`), order *Dark Matter*, say you love sci-fi, and then type `quit`. **Session 2 (new process, fresh memory)** — You run the script again as `mohit_47` and say "Hey there!". MannKiBot correctly greets you, recalls the books you ordered, and remembers your genre preferences — all from ChromaDB.

2. Core Concepts

2.1 Why Memory Matters

Every LLM call is stateless. The model has no idea what was said in a previous message unless you include it in the current request. All "memory" in LLM applications is context management — deciding what text to include in the prompt.

The challenge is the **context window limit**. You can't keep appending every message forever. A 100-turn conversation would eventually be too long and too expensive to send with every new message. This project implements three strategies to handle this.

2.2 The Three Memory Layers

MEMORY ARCHITECTURE		
Layer 1	Layer 2	Layer 3
Sliding Window	Summary Memory	Long-Term (ChromaDB)
Last 10 msgs	LLM-compressed	Embedded facts stored
Lives in RAM	older messages	on disk as vectors
Dies on exit	Lives in RAM	Persists forever
	Dies on exit	
In-session	In-session	Cross-session
Short-term	Short-term	Long-term

Layer 1 — Sliding Window Buffer Always expose only the most recent N messages (default: 10) to the LLM. When message 11 arrives, message 1 is dropped. Simple and cheap — no extra LLM calls needed.

Layer 2 — Summary Memory Instead of silently dropping old messages, call the LLM to compress them into 2-3 sentences before they fall out of the window. That summary is prepended to future prompts as context. This preserves semantic continuity at the cost of one extra LLM call per overflow event.

Layer 3 — ChromaDB Long-Term Memory After a session ends, extract key facts (name, orders, preferences) using an LLM call and store them as vector embeddings in ChromaDB. At the start of the next session, retrieve semantically relevant facts and inject them into the system prompt. This is the only layer that survives application restarts.

2.3 Memory Strategy Comparison

Strategy	Cost	What's Preserved	When It Fails
Buffer memory (store everything)	High tokens	Everything	Context window overflow
Sliding window	Low tokens	Recent context only	Loses early facts
Summary memory	Medium tokens	Semantic gist	Nuance lost in compression
Long-term (ChromaDB)	Disk + embedding	Key facts across sessions	Only as good as extraction

3. Project Files Overview

The codebase is highly modular, splitting responsibilities cleanly:

```
Problem6b/
├─ config.py           # Constants, system prompts, Ollama HTTP checks
├─ memory.py          # ChromaDB vector storage and extraction
├─ graph.py           # LangGraph nodes, state machine, and router
├─ session.py         # BookstoreSession wrapper over the graph
├─ main.py            # The interactive CLI loop
├─ chroma_bookstore_ollama/ # Created at runtime (database folder)
```

4. The Modular Implementation

Here is a highly detailed, line-by-line breakdown of the critical code blocks that power the project.

4.1 `config.py` - Configuration and LLM Interface

This file centralizes all global constants, system prompts, and handles the direct HTTP interactions with the local Ollama LLM.

- **Checking Ollama Readiness:** Before the application can run, it verifies that the Ollama service is active and has the correct models.

```
def check_ollama(required_models: list[str]) -> bool:
    # Pings the local Ollama API to see if the daemon is running
    r = requests.get(f"{OLLAMA_BASE_URL}/api/tags", timeout=5)
    r.raise_for_status()

    # Extracts the names of the models currently downloaded
    pulled = {m["name"].split(":")[0] for m in r.json().get("models", [])}

    # Checks if both llama3.1 (chat) and nomic-embed-text (embeddings) are present
    missing = [m for m in required_models if m.split(":")[0] not in pulled]
    if missing: return False
    return True
```

- **The Core LLM Wrapper (`_ollama_complete`)**: LangChain's `ChatOllama` object expects an array of Message objects (`SystemMessage`, `HumanMessage`, `AIMessage`). This function constructs that array.

```
def _ollama_complete(system: str, history: list[AnyMessage]) -> str:
    # Prepend the system instructions to the start of the chat history
    msgs = [SystemMessage(content=system)] + list(history)

    # Invoke the model. This is a blocking call that generates the response text
    response = _llm.invoke(msgs)

    # Extract just the raw text string from the AIMessage object
    return response.content.strip()
```

4.2 `memory.py` - Persistent Vector Storage (Layer 3)

This module handles ChromaDB, the local vector database used for persisting data across multiple sessions.

- **Adapter for Embeddings (`OllamaChromaEF`)**: ChromaDB requires an `EmbeddingFunction` class to know how to turn text into vectors. We create a custom class that wraps `langchain_ollama.OllamaEmbeddings` so ChromaDB knows to ping Ollama for the embeddings.

```

class OllamaChromaEF(EmbeddingFunction):
    def __init__(self, model=EMBED_MODEL, base_url=OLLAMA_BASE_URL):
        self._emb = OllamaEmbeddings(model=model, base_url=base_url)

    def __call__(self, input: Documents) -> Embeddings:
        # Converts a list of text strings into a list of vector arrays
        return self._emb.embed_documents(list(input))

```

- **Retrieval & Storage (LongTermMemory):** The memory object initializes the database locally in the project directory.

```

def retrieve(self, user_id: str, query: str = "...", n: int = 3) -> str:
    # We query the collection using semantic search.
    # The where={"user_id": user_id} ensures we ONLY search this specific user's memories!
    results = self.col.query(
        query_texts=[query],
        n_results=n,
        where={"user_id": user_id}
    )
    # Then we extract and format the resulting strings
    ...

def store(self, user_id: str, facts: str, session_id: str):
    # Upsert is critical: it overwrites if the ID exists, or creates if it doesn't.
    # The ID is deterministic (user + session), preventing duplicates.
    doc_id = f"{user_id}_{session_id}"
    self.col.upsert(
        ids=[doc_id],
        documents=[facts], # The plain english facts generated by the LLM
        metadatas=[{"user_id": user_id, "session_id": session_id}]
    )

```

4.3 graph.py - LangGraph Flow and Logic (The State Machine)

LangGraph manages the immediate context window (Layer 1) and the dynamic summary memory (Layer 2).

- **Defining the State:** The AgentState defines the data that flows through the graph. The add_messages reducer tells LangGraph how to update the message list (e.g., appending new ones or removing deleted ones).

```
class AgentState(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]
    summary: str
    user_id: str
    session_id: str
    lt_context: str
    turn_count: int
```

- **Node: Chat (Layer 1 - Sliding Window):** This function executes every single time the user sends a message.

```
def node_chat(state: AgentState) -> dict:
    messages = state["messages"]
    summary = state.get("summary", "")

    # 1. We start by building the context window.
    window_msgs = []

    # 2. If Layer 2 (Summary) exists, we inject it synthetically as a fake Human/AI pair
    if summary:
        window_msgs.append(HumanMessage(content="[System note: earlier summary]"))
        window_msgs.append(AIMessage(content=summary))

    # 3. Sliding Window Strategy: Only take the last 10 messages
    window_msgs.extend(messages[-WINDOW_SIZE:])

    # 4. Generate the reply using the LLM
    system = build_system_prompt(state.get("lt_context", ""))
    reply = _ollama_complete(system, window_msgs)

    # 5. Return the new message to be appended to the state
    return {"messages": [AIMessage(content=reply)]}
```

- **Node: Summarise (Layer 2 - Semantic Compression):** If the sliding window overflows, this node compresses the old data.

```
def node_summarise(state: AgentState) -> dict:
    # Calculate how many messages are over the limit
    overflow_count = len(messages) - WINDOW_SIZE

    # Slice out the old messages that are about to be dropped
    to_compress = messages[:overflow_count]

    # Prompt the LLM to compress them into 2-3 sentences
    summarise_system = "You are a precise summariser... Output ONLY a 2-3 sentence summary."
    new_piece = _ollama_complete(summarise_system, [...])

    # Crucially: Return RemoveMessage objects. LangGraph sees these and deletes
    # the old, uncompressed messages from RAM to save tokens.
    removals = [RemoveMessage(id=m.id) for m in to_compress]

    return {"summary": new_piece, "messages": removals}
```

- **The Router:** The conditional logic that triggers compression.

```
def router_should_summarise(state: AgentState) -> Literal["summarise", "__end__"]:
    if len(state["messages"]) > WINDOW_SIZE:
        return "summarise"
    return "__end__"
```

- **Building the Graph:** This binds all the nodes and edges into an executable application, utilizing a `MemorySaver` checkpointer to hold the state in RAM during the active session.

```
def build_graph(ltm: LongTermMemory):
    g = StateGraph(AgentState)
    g.add_node("load_ltm", _load_ltm)
    g.add_node("chat", node_chat)
    g.add_node("summarise", node_summarise)

    g.set_entry_point("load_ltm")
    g.add_edge("load_ltm", "chat")

    # Conditional edge evaluating if we need to summarize before ending the turn
    g.add_conditional_edges("chat", router_should_summarise)
    g.add_edge("summarise", END)

    return g.compile(checkpointer=MemorySaver())
```

4.4 `session.py` - Interaction Abstraction

This file bridges the gap between the complex LangGraph execution and the simple CLI loop.

- **Session Isolation (`thread_id`)**: LangGraph tracks state using a `thread_id`. If user A and user B have the same thread ID, they share the same context window.

```
class BookstoreSession:
    def __init__(self, app, ltm, user_id: str):
        # We attach a random UUID suffix.
        # This guarantees that if "mohit" logs out and logs back in, he gets
        # a fresh short-term memory slate, preventing previous context from bleeding
        in.

        self.thread_id = f"{user_id}_{str(uuid.uuid4())[:8]}"
```

- **Ending the Session (`end()`)**: When the user closes the app, we need to save the short-term memories into Layer 3 (ChromaDB).


```

def end(self):
    # Fetch the full state as it exists right now in RAM
    snapshot = self.app.get_state({"configurable": {"thread_id": self.thread_id}})

    state = snapshot.values

    # Call the extract_facts LLM helper (from graph.py) to read the whole conversation
    # and output plain English facts (e.g. "User likes sci-fi").
    facts = extract_facts(state["messages"], state.get("summary", ""))

    # Save those prose facts to the database
    if facts:
        self.ltm.store(self.user_id, facts, self.session_id)

```

4.5 `main.py` - The User Interface

The entry point of the script handles the continuous interaction loop.

- `run_interactive()`: An infinite loop utilizing Python's `input()`. The most vital part is the `finally` block. Because `session.end()` must run to save memories to ChromaDB, the `try/finally` block guarantees that even if the user hits `Ctrl+C` to kill the script, the memory extraction still runs.

```

try:
    while True:
        user_input = input("👤 You: ").strip()
        reply = session.chat(user_input)
        print(f"\n🤖 MannKiBot: {reply}\n")
finally:
    # Guaranteed to run on exit/crash!
    print("\nEnding session and saving memories...")
    if 'session' in locals():
        session.end()

```

5. The Interactive Demo Explained

When you run `python main.py`, the CLI asks for your username.

1. Entering your username initializes `BookstoreSession`.
2. The agent queries ChromaDB. If you are new, it proceeds normally. If you return, it invisibly injects your previous history into `lt_context`.

3. You converse. If your conversation gets long (over `WINDOW_SIZE=10`), the agent automatically compresses the oldest messages, preserving tokens.
4. When you type `quit`, the loop breaks, triggering the `finally` block. The agent reads the graph state, extracts facts into prose, and saves them permanently.

6. Key Design Decisions

Why prose for fact storage, not JSON?

Facts are stored as plain English sentences rather than a JSON object. Prose embeddings carry richer semantic meaning. When you query ChromaDB with "what did this user order", the cosine similarity between the query embedding and a prose embedding of facts is more reliable than the similarity to a JSON string. JSON structure can interfere with embedding quality.

Why upsert instead of add for ChromaDB?

```
self.collection.upsert(ids=[doc_id], ...)
```

Using `upsert` means re-running the same session won't create duplicate entries. The `doc_id` format (`user_id_session_id`) is deterministic, so the same session re-run just overwrites the previous entry. `add` would throw an error on the second run because the ID already exists.

Why a synthetic Human/AI pair for the summary injection?

```
api_messages.append({"role": "user", "content": "[System note: earlier summary]"})
api_messages.append({"role": "assistant", "content": f"[Summary]: {summary}"})
```

Most LLMs expect messages to alternate between human and assistant roles. Injecting the summary as a single system-like message or a bare assistant message can cause issues. Framing it as a completed exchange (question + answer) is reliably interpreted as shared context from earlier in the conversation.

Why `RemoveMessage` instead of returning a shorter list?

In LangGraph, the `add_messages` reducer is append-only by design. If you return `{"messages": short_list}`, the reducer tries to append `short_list` to the existing list, not replace it. `RemoveMessage` objects are the proper way to signal deletions — the reducer recognises them and deletes the matching message by ID before merging.

Why per-session `thread_id` instead of per-user?

```
self.thread_id = f"{user_id}_{self.session_id}"
```

Using the user ID directly as `thread_id` would cause Session 2 to continue loading the in-session state from Session 1 (via MemorySaver). A new UUID suffix means each session starts with a clean in-memory slate. Cross-session memory comes from ChromaDB, not from the MemorySaver — this separation is intentional.

7. Extending the Project

Swap the embedding model

To improve retrieval quality, swap to a larger embedding model:

```
ollama pull mxbai-embed-large # 1024-dim, higher quality
```

In `config.py`:

```
EMBED_MODEL = "mxbai-embed-large"
```

Then delete the old ChromaDB directory (embedding dimensions must match) and re-run.

Add more LTM collections

Instead of one collection for all user facts, use separate collections for different memory types:

```
orders_col = chroma.get_or_create_collection("orders")
preferences_col = chroma.get_or_create_collection("preferences")
```

8. Troubleshooting

```
Cannot reach Ollama at http://localhost:11434
```

Ollama isn't running. Start it:

```
ollama serve # Linux/macOS foreground
```

Missing Ollama models

```
ollama pull llama3.1
ollama pull nomic-embed-text
```

ChromaDB embedding dimension mismatch

```
InvalidDimensionException: Embedding dimension X does not match collection dimensionality Y
```

You changed the embedding model but the existing collection was built with the old one. Delete it:

```
rm -rf ./chroma_bookstore_ollama/
python3 main.py          # re-run to rebuild
```

Session 2 doesn't recall Session 1 facts

Check in order:

1. Did Session 1 complete without errors? The `quit` command must be run properly.
2. Is `user_id` the exactly same in both sessions? (e.g. `"mohit_47"`)
3. Does the ChromaDB directory exist?
4. Add a debug print before querying: `print(ltm.col.count())` — should be ≥ 1 after Session 1.